

GhostHouses

Stage B Presentation

GhostHouses Team

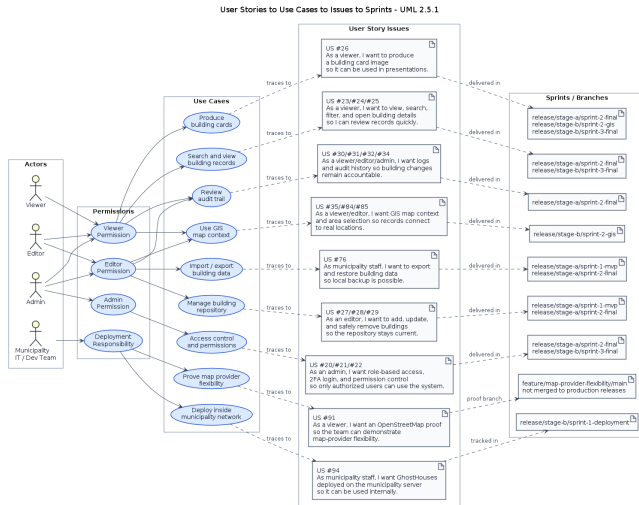
Technion, Faculty of Computer Science

2025–2026 Yearly Project in Software Engineering

Agenda

- 1 User Stories to Use Cases to Issues to Sprints
- 2 Architecture
- 3 Customer-Side Deployment
- 4 Open Discussion

Traceability UML

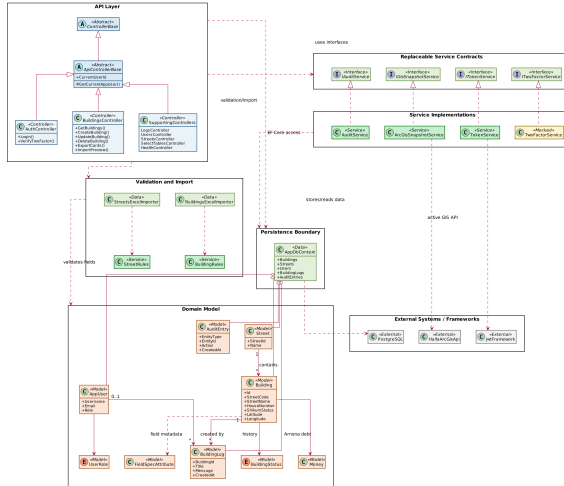


User Stories to Use Cases to Issues to Sprints

- We grouped the backlog by actors, permission levels, and real user-facing use cases.
- Each use case maps to readable GitHub User Story issues, not only issue numbers.
- The diagram keeps implementation sub-issues out of the slide so the presentation stays readable.
- Release branches capture sprint checkpoints: Stage A MVP, Stage A final, Stage B deployment, Stage B GIS, and the final handoff state.

Class UML

GhostHouses Maintainable Class Design - UML 2.5.1



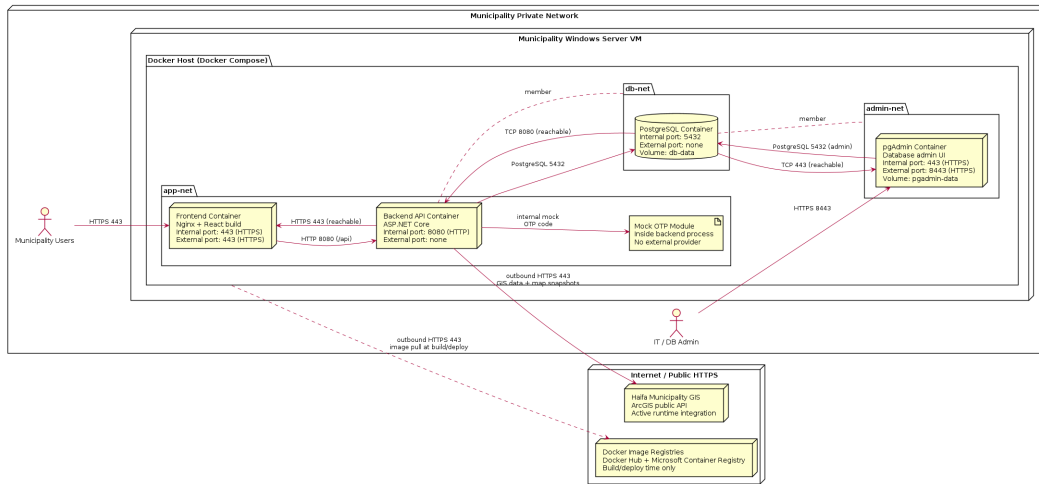
Architecture: Abstract vs. Concrete

- Abstract boundaries: service contracts for JWT, OTP, audit logging, and GIS snapshots.
- Concrete commitments: ASP.NET Core backend, React frontend, PostgreSQL database, ArcGIS GIS provider, and Docker Compose deployment.
- The external-risk areas stay behind interfaces, so OTP delivery or GIS provider changes do not require rewriting controllers or the domain model.
- Stable runtime choices are concrete because they define how the municipality receives and operates the system.

- 'ApiControllerBase' centralizes authenticated API behavior.
- 'BuildingRules' and 'StreetRules' keep manual editing and Excel import aligned to one source of truth.
- 'AppDbContext' is the persistence boundary for PostgreSQL entities.
- 'IGisSnapshotService' protects building-card export from a future GIS vendor change.

Deployment UML

GhostHouses Stage B Deployment Environment - UML 2.5.1



Customer-Side Deployment

- The municipality environment is a Windows Server VM inside the private municipality network.
- The intended deployment is a Docker Compose stack: frontend, backend, PostgreSQL, and pgAdmin.
- Municipality users access only the frontend through HTTPS 443.
- IT/database administration uses pgAdmin through HTTPS 8443.
- Backend and database stay internal to Docker networks and are not exposed directly.

Deployment Woes and Mitigation

- Main blocker: Docker Desktop / WSL 2 on the municipality Windows Server VM may require nested virtualization approval.
- We prepared the deployment as one repeatable command from `project/`: `docker compose down -v && docker compose up -d --build`.
- We documented ports, software dependencies, hardware expectations, secrets, certificates, and network access.
- We prepared fallback plans with the supervisor if nested virtualization is not approved.
- Deployment communication is documented in writing with the client side, municipality dev team, IT/security contacts, and supervisors.

Questions for the Audience

- TODO: Add the real technical/process question we want feedback on.